

Module 3

1. Control Structures

Code:

Creating the Tables first:

```
CREATE TABLE Customers (CustomerID NUMBER PRIMARY KEY, Name  
VARCHAR2(50), Age NUMBER, Balance NUMBER, IsVIP CHAR(1));
```

```
CREATE TABLE Loans (LoanID NUMBER PRIMARY KEY, CustomerID NUMBER,  
InterestRate NUMBER(5,2), DueDate DATE, FOREIGN KEY(CustomerID) REFERENCES  
Customers(CustomerID));
```

```
INSERT INTO Customers VALUES (1,'John',65,15000,'N');  
INSERT INTO Customers VALUES (2,'Alice',35,8000,'N');  
INSERT INTO Customers VALUES (3,'Robert',70,12000,'N');  
INSERT INTO Customers VALUES (4,'Emma',55,20000,'N');
```

```
INSERT INTO Loans VALUES (101,1,10.5,SYSDATE+20);  
INSERT INTO Loans VALUES (102,2,11.0,SYSDATE+40);  
INSERT INTO Loans VALUES (103,3,9.5,SYSDATE+10);  
INSERT INTO Loans VALUES (104,4,12.0,SYSDATE+25);
```

```
COMMIT;
```

Scenario 1

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
BEGIN
```

```
FOR rec IN (SELECT l.LoanID, c.Age, l.InterestRate FROM Customers c JOIN Loans  
l ON c.CustomerID=l.CustomerID)
```

```
LOOP
```

```
IF rec.Age > 60 THEN
```

```
UPDATE Loans
```

```
SET InterestRate = InterestRate - 1
```

```
WHERE LoanID = rec.LoanID;
```

```
DBMS_OUTPUT.PUT_LINE('Loan ID ' || rec.LoanID || ' updated. New Interest  
Rate = ' || (rec.InterestRate-1));
```

```

END IF;

END LOOP;

COMMIT;

END;

/

```

Output:

Query result Script output DBMS output Explain Plan SQL history				
  Download ▼ Execution time: 0.006 seconds				
	LOANID	CUSTOMERID	INTERESTRATE	DUE DATE
1	101	1	9.5	8/9/2026, 6:42:52 AM
2	102	2	11	8/29/2026, 6:42:52 AM
3	103	3	8.5	7/30/2026, 6:42:52 AM
4	104	4	12	8/14/2026, 6:42:52 AM

Scenario 2:

```

SET SERVEROUTPUT ON;

DECLARE

BEGIN

FOR rec IN (SELECT CustomerID, Balance FROM Customers)

LOOP

IF rec.Balance > 10000 THEN

UPDATE Customers

SET IsVIP='Y'

WHERE CustomerID=rec.CustomerID;

DBMS_OUTPUT.PUT_LINE('Customer '|| rec.CustomerID|| ' promoted to VIP.');

```

/

Output:

```
Query result  Script output  DBMS output  Explain Plan  SQL history

[Icons: Trash, Download]

FOR rec IN (SELECT customerid, balance FROM customers)
LOOP...
Show more...

Customer 1 promoted to VIP.
Customer 3 promoted to VIP.
Customer 4 promoted to VIP.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.156
```

Query result Script output DBMS output Explain Plan SQL history					
[Icons: Trash, Info] Download ▾ Execution time: 0.004 seconds					
	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	John	65	15000	Y
2	2	Alice	35	8000	N
3	3	Robert	70	12000	Y
4	4	Emma	55	20000	Y

Scenario 3:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
BEGIN
```

```
    FOR rec IN (SELECT c.Name,l.LoanID,l.DueDate FROM Customers c JOIN Loans l
ON c.CustomerID=l.CustomerID WHERE l.DueDate BETWEEN SYSDATE AND
SYSDATE+30)
```

```
    LOOP
```

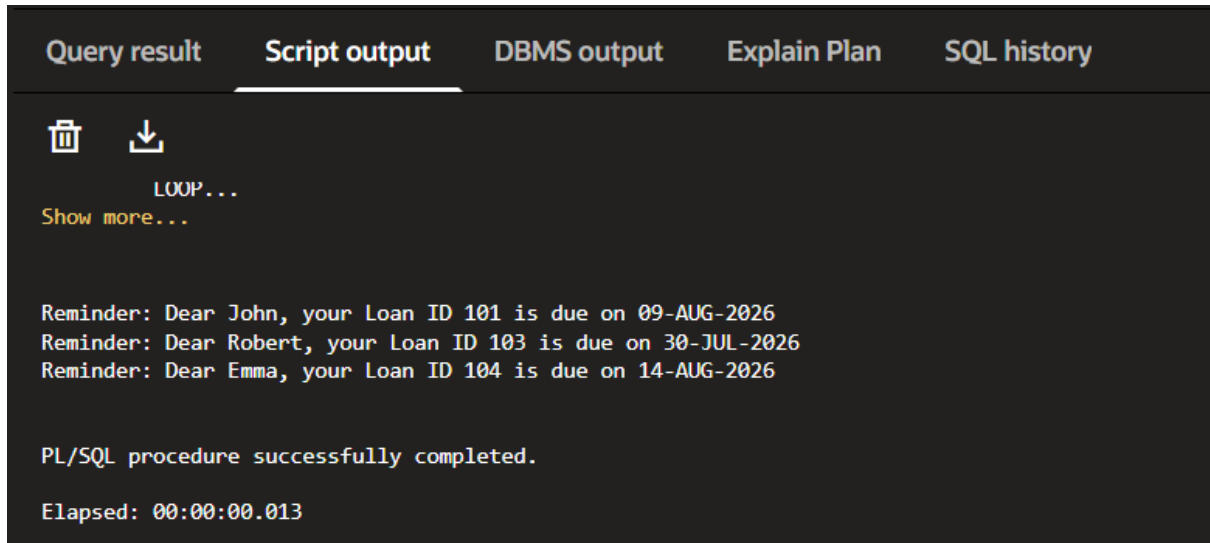
```
        DBMS_OUTPUT.PUT_LINE('Reminder: Dear '|| rec.Name|| ', your Loan ID '||
rec.LoanID|| ' is due on '|| TO_CHAR(rec.DueDate,'DD-MON-YYYY'));
```

```
    END LOOP;
```

```
END;
```

/

Output:

A screenshot of a database application window with a dark theme. The window has a tabbed interface with five tabs: 'Query result', 'Script output' (which is selected and underlined), 'DBMS output', 'Explain Plan', and 'SQL history'. Below the tabs, there are two icons: a trash can and a download arrow. The main area displays the output of a script. It starts with 'LOOP...' followed by 'Show more...'. Then, it shows three reminder messages: 'Reminder: Dear John, your Loan ID 101 is due on 09-AUG-2026', 'Reminder: Dear Robert, your Loan ID 103 is due on 30-JUL-2026', and 'Reminder: Dear Emma, your Loan ID 104 is due on 14-AUG-2026'. Below these, it says 'PL/SQL procedure successfully completed.' and 'Elapsed: 00:00:00.013'.

2. Stored Procedures

Code:

-- Customers Table

```
CREATE TABLE Customers (CustomerID NUMBER PRIMARY KEY, CustomerName  
VARCHAR2(50));
```

-- Accounts Table

```
CREATE TABLE Accounts (AccountID NUMBER PRIMARY KEY, CustomerID  
NUMBER, AccountType VARCHAR2(20), Balance NUMBER(12,2), FOREIGN  
KEY(CustomerID) REFERENCES Customers(CustomerID));
```

-- Employees Table

```
CREATE TABLE Employees (EmployeeID NUMBER PRIMARY KEY, EmployeeName  
VARCHAR2(50), Department VARCHAR2(30), Salary NUMBER(10,2));
```

```
INSERT INTO Customers VALUES (1,'John');
```

```
INSERT INTO Customers VALUES (2,'Alice');
```

```
INSERT INTO Customers VALUES (3,'Robert');
```

```
INSERT INTO Accounts VALUES (101,1,'Savings',10000);
```

```
INSERT INTO Accounts VALUES (102,2,'Savings',25000);
```

```
INSERT INTO Accounts VALUES (103,3,'Current',15000);
```

```
INSERT INTO Accounts VALUES (104,1,'Savings',5000);
```

```
INSERT INTO Employees VALUES (1,'Rahul','IT',50000);
```

```
INSERT INTO Employees VALUES (2,'Priya','HR',45000);
```

```
INSERT INTO Employees VALUES (3,'Amit','IT',60000);
```

```
INSERT INTO Employees VALUES (4,'Neha','Finance',55000);
```

```
COMMIT;
```

Output:

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.146 seconds					
	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	John	65	15000	Y
2	2	Alice	35	8000	N
3	3	Robert	70	12000	Y
4	4	Emma	55	20000	Y

Scenario 1:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
```

```
AS
```

```
BEGIN
```

```
    UPDATE Accounts
```

```
    SET Balance = Balance + (Balance * 0.01)
```

```
    WHERE AccountType='Savings';
```

```
    COMMIT;
```

```
END;
```

```
/
```

Output:

Query result Script output DBMS output Explain Plan SQL history				
  Download ▾ Execution time: 0.009 seconds				
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	101	1	Savings	10000
2	102	2	Savings	25000
3	103	3	Current	15000
4	104	1	Savings	5000

Scenario 2:

```
SELECT * FROM Accounts;
```

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(p_department IN  
VARCHAR2, p_bonus IN NUMBER)
```

```
AS
```

```
BEGIN
```

```
    UPDATE Employees
```

```
    SET Salary = Salary + (Salary * p_bonus/100)
```

```
    WHERE Department = p_department;
```

```
    COMMIT;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    UpdateEmployeeBonus('IT',10);
```

```
END;
```

```
/
```

```
SELECT * FROM Employees;
```

Output:

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.01 seconds					
	EMPLOYEEID	EMPLOYEE NAME	DEPARTMENT	SALARY	
1	1	Rahul	IT	55000	
2	2	Priya	HR	45000	
3	3	Amit	IT	66000	
4	4	Neha	Finance	55000	

Scenario 3:

```
CREATE OR REPLACE PROCEDURE TransferFunds(p_fromAccount IN NUMBER,  
p_toAccount IN NUMBER, p_amount IN NUMBER)
```

```
AS
```

```
    v_balance NUMBER;
```

```
BEGIN
```

```
    SELECT Balance
```

```
    INTO v_balance
```

```
    FROM Accounts
```

```
    WHERE AccountID=p_fromAccount;
```

```
    IF v_balance >= p_amount THEN
```

```
        UPDATE Accounts
```

```
        SET Balance=Balance-p_amount
```

```
        WHERE AccountID=p_fromAccount;
```

```
    UPDATE Accounts
```

```
    SET Balance=Balance+p_amount
```

```
    WHERE AccountID=p_toAccount;
```

```
    COMMIT;
```

```
    DBMS_OUTPUT.PUT_LINE('Transfer Successful');
```

```

ELSE
    DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
END IF;
END;
/
SET SERVEROUTPUT ON;
BEGIN
    TransferFunds(101,102,3000);
END;
/
SELECT * FROM Accounts;

```

Output:

Query result Script output DBMS output Explain Plan SQL history					
  Download ▼ Execution time: 0.001 seconds					
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	
1	101	1	Savings	7000	
2	102	2	Savings	28000	
3	103	3	Current	15000	
4	104	1	Savings	5000	